



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

**IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED
REASONING (RBR), AND CASE-BASED REASONING (CBR)**

Student Name	Dianna Francesca M. Pardilla Justin D. Vergara
Section:	AN41
Professor:	Ms. Crisola G. Tan
Date:	Oct 3, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge.

Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware

- A laptop or desktop computer
- Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
(Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {  
    "temperature": 32,  
    "is_raining": False,  
    "humidity": 85,  
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF-THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):  
    if facts["temperature"] > 30:  
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:

- Turn on the aircon
- Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [  
    {"problem": {"won't turn on": True, "noise": False},  
     "solution": "Check power cable."}  
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks

```
import os
import sys
import time
from typing import Dict, List, Any, Tuple

# TERMINAL UI & AESTHETICS (BUTTON STYLING + ALIGNED BORDERS)
class UI:
    CYAN = "\033[96m"
    GREEN = "\033[92m"
    YELLOW = "\033[93m"
    MAGENTA = "\033[95m"
    RED = "\033[91m"
    BOLD = "\033[1m"
    DIM = "\033[2m"
    RESET = "\033[0m"

    # Inner width between the vertical pipes: | ... 74 chars ... |
    INNER_WIDTH = 74

    @classmethod
    def header(cls) -> None:
        cls.clear()
        w = cls.INNER_WIDTH

        top_border = f"{cls.CYAN}+{'-' * w}+{cls.RESET}"
        mid_border = f"{cls.CYAN}+{'-' * w}+{cls.RESET}"
        bot_border = f"{cls.CYAN}+{'-' * w}+{cls.RESET}"

        # Plain text strings for exact width calculation
        t1_plain = "INTELLIGENT SYSTEMS: REASONING & INFERENCE ENGINE"
        t2_plain = "CS0065 - Technical Assessment 4"
        l3_plain = "    Programmed by : Dianna Francesca M. Pardilla & Justin D. Vergara"
        l4_plain = "    Section          : CS0065 - AN41"

        # Centered titles
```

```

        t1_pad = (w - len(t1_plain)) // 2
        t1_formatted = " " * t1_pad + f"{cls.BOLD}{t1_plain}{cls.RESET}" +
" " * (w - len(t1_plain) - t1_pad)

        t2_pad = (w - len(t2_plain)) // 2
        t2_formatted = " " * t2_pad + f"{cls.DIM}{t2_plain}{cls.RESET}" +
" " * (w - len(t2_plain) - t2_pad)

        # Left-aligned info lines with exact trailing space padding
        l3_formatted = f"    Programmed by : {cls.BOLD}Dianna Francesca M.
Pardilla & Justin D. Vergara{cls.RESET}" + " " * (w - len(l3_plain))
        l4_formatted = f"    Section                : {cls.YELLOW}CS0065 -
AN41{cls.RESET}" + " " * (w - len(l4_plain))

        # Render Header Box
        print(top_border)

print(f"{cls.CYAN}|{cls.RESET}{t1_formatted}{cls.CYAN}|{cls.RESET}")

print(f"{cls.CYAN}|{cls.RESET}{t2_formatted}{cls.CYAN}|{cls.RESET}")
    print(mid_border)

print(f"{cls.CYAN}|{cls.RESET}{l3_formatted}{cls.CYAN}|{cls.RESET}")

print(f"{cls.CYAN}|{cls.RESET}{l4_formatted}{cls.CYAN}|{cls.RESET}")
    print(bot_border)

    @classmethod
    def section(cls, title: str, part_num: str) -> None:
        """Displays section headers with a stylized < BUTTON > banner."""
        tag_btn      = f"{cls.MAGENTA}<
{cls.BOLD}{part_num}{cls.RESET}{cls.MAGENTA} >{cls.RESET}"
        title_btn    = f"{cls.CYAN}<
{cls.BOLD}{title.upper()}{cls.RESET}{cls.CYAN} >{cls.RESET}"
        print(f"\n[tag_btn] — {title_btn}\n")

    @classmethod

```

```

def button(cls, key: str, label: str) -> str:
    """Returns a formatted button string for menu choices."""
    badge = f"{cls.CYAN}< {cls.BOLD}{key}{cls.RESET}{cls.CYAN}>{cls.RESET}"
    return f" {badge} {label}"

@classmethod
def pause(cls) -> None:
    print(f"\n{cls.DIM}Press Enter to return to main menu...{cls.RESET}", end="")
    input()

@classmethod
def clear(cls) -> None:
    os.system("cls" if os.name == "nt" else "clear")

# PART 1: KNOWLEDGE REPRESENTATION (KR) SETUP
knowledge_base: Dict[str, Any] = {
    # Category 1: Thermals & Environment
    "core_temperature_celsius": 86,
    "ambient_humidity_percent": 74,

    # Category 2: Resource Utilization & Power
    "cpu_load_percent": 96,
    "cooling_pump_active": False,
    "battery_charge_percent": 15,

    # Category 3: System Health & Hardware Flags
    "disk_io_errors": True,
    "network_latency_ms": 350
}

def display_knowledge_base() -> None:
    UI.header()
    UI.section("Knowledge Representation (Working Memory)", "PART 1")

```

```

        print(f"      {UI.BOLD}{'Telemetry  Parameter':<32} | {'State  /
Value'}{UI.RESET}")
    print("    " + "-" * 56)

    for key, value in knowledge_base.items():
        label = key.replace("_", " ").title()
        if isinstance(value, bool):
            val_display = f"{UI.RED}< CRITICAL: True >{UI.RESET}" if value
        else f"{UI.DIM}< NORMAL: False >{UI.RESET}"
            elif isinstance(value, (int, float)) and ("temp" in key or "load"
in key):
                val_display = f"{UI.YELLOW}< {value} >{UI.RESET}"
            else:
                val_display = f"{UI.GREEN}< {value} >{UI.RESET}"

        print(f"    • {label:<30} | {val_display}")

    UI.pause()

# PART 2: RULE-BASED REASONING (RBR)
def rule_thermal_critical(facts: Dict[str, Any]) -> str | None:
    if facts.get("core_temperature_celsius", 0) > 80:
        return "Thermal overload (>80°C) -> Trigger immediate CPU clock
down-throttling."
    return None

def rule_cooling_failure(facts: Dict[str, Any]) -> str | None:
    if facts.get("core_temperature_celsius", 0) > 75 and not
facts.get("cooling_pump_active", True):
        return "Cooling pump offline under high load -> Force auxiliary
high-RPM fan bypass."
    return None

def rule_battery_preservation(facts: Dict[str, Any]) -> str | None:
    if facts.get("battery_charge_percent", 100) < 20:
        return "Reserve power depleted (<20%) -> Terminate secondary GPU
threads and background tasks."

```

```

    return None

def rule_disk_anomaly(facts: Dict[str, Any]) -> str | None:
    if facts.get("disk_io_errors", False):
        return "I/O integrity flags asserted -> Mount main file-system
partition as read-only."
    return None

class InferenceEngine:
    def __init__(self, rule_list: List[Any]):
        self.rules = rule_list

    def run(self, facts: Dict[str, Any]) -> List[Tuple[str, str]]:
        deductions = []
        for i, r in enumerate(self.rules, start=1):
            result = r(facts)
            if result:
                deductions.append((f"RBR-{i:02d}", result))
        return deductions

def run_rule_based_reasoning() -> None:
    UI.header()
    UI.section("Rule-Based Reasoning (Inference Cycle)", "PART 2")

    engine = InferenceEngine([
        rule_thermal_critical,
        rule_cooling_failure,
        rule_battery_preservation,
        rule_disk_anomaly
    ])

    print(f"    {UI.DIM}[*] Evaluating working memory across defined
production rules...{UI.RESET}")
    time.sleep(0.3)
    actions = engine.run(knowledge_base)

```

```

        print(f"\n    {UI.BOLD}Inferred Prescriptive Actions ({len(actions)}
matched):{UI.RESET}\n")
        for rule_id, act in actions:
            btn_tag = f"{UI.YELLOW}< {rule_id} >{UI.RESET}"
            print(f"    {btn_tag} {act}")

    UI.pause()

# PART 3: CASE-BASED REASONING (CBR) - 4R CYCLE
case_base: List[Dict[str, Any]] = [
    {
        "id": "CASE-101",
        "problem": {"overheating": True, "fan_spinning": False,
"loud_buzzing": False, "blue_screen": False},
        "solution": "Clean heatsink fins and replace blocked auxiliary fan
motor."
    },
    {
        "id": "CASE-102",
        "problem": {"overheating": True, "fan_spinning": True,
"loud_buzzing": True, "blue_screen": False},
        "solution": "Inspect fan blades for physical friction; lubricate
bearing housing."
    },
    {
        "id": "CASE-103",
        "problem": {"overheating": False, "fan_spinning": True,
"loud_buzzing": False, "blue_screen": True},
        "solution": "Run memory diagnostic tool (memtest86) to detect
degraded RAM modules."
    },
    {
        "id": "CASE-104",
        "problem": {"overheating": True, "fan_spinning": False,
"loud_buzzing": True, "blue_screen": True},

```

```

        "solution": "Power supply unit (PSU) rail short-circuit; perform
full board inspection."
    }
]

def compute_similarity(stored_problem: Dict[str, bool], incoming_problem:
Dict[str, bool]) -> float:
    keys = set(stored_problem.keys()) & set(incoming_problem.keys())
    if not keys:
        return 0.0
    matches = sum(1 for k in keys if stored_problem[k] ==
incoming_problem[k])
    return (matches / len(keys)) * 100.0

def retrieve(incoming: Dict[str, bool]) -> Tuple[Dict[str, Any], float]:
    best_match = None
    highest_score = -1.0
    for case in case_base:
        score = compute_similarity(case["problem"], incoming)
        if score > highest_score:
            highest_score = score
            best_match = case
    return best_match, highest_score

def run_case_based_reasoning() -> None:
    UI.header()
    UI.section("Case-Based Reasoning (4R Cycle)", "PART 3")

    incoming_case = {
        "overheating": True,
        "fan_spinning": False,
        "loud_buzzing": False,
        "blue_screen": True
    }

```

```

print(f"    {UI.BOLD}Incoming Problem Vector:{UI.RESET}")
for k, v in incoming_case.items():
    print(f"        • {k:<15}: {v}")

# Step 1: RETRIEVE
print(f"\n    {UI.CYAN}< 1. RETRIEVE >{UI.RESET} Searching case library
for closest pattern...")
time.sleep(0.3)
matched_case, score = retrieve(incoming_case)
print(f"    -> Best Match : {UI.BOLD}< {matched_case['id']} >{UI.RESET}
(Similarity: {UI.GREEN}{score:.1f}%{UI.RESET})")

# Step 2: REUSE
print(f"\n    {UI.CYAN}< 2. REUSE >{UI.RESET} Retrieved Solution:")
reused_solution = matched_case["solution"]
print(f"    -> \"{reused_solution}\"")

# Step 3: REVISE
print(f"\n    {UI.CYAN}< 3. REVISE >{UI.RESET} Adjusting solution for
unique symptom traits:")
print("    -> Enter adjusted solution (press Enter to keep default): ")
user_adj = input("        > ").strip()
revised_solution = user_adj if user_adj else f"{reused_solution} +
[Addendum: Verify PSU voltage output]."
print(f"    -> Final Solution: {UI.YELLOW}{revised_solution}{UI.RESET}")

# Step 4: RETAIN
print(f"\n    {UI.CYAN}< 4. RETAIN >{UI.RESET} Archiving resolved case
into memory base...")
new_id = f"CASE-{len(case_base) + 101}"
learned_case = {
    "id": new_id,
    "problem": incoming_case,
    "solution": revised_solution
}
case_base.append(learned_case)

```

```

    print(f"    -> Case registered as {UI.BOLD}< {new_id} >{UI.RESET}.")
        print(f"                ->      Total      stored      cases:
{UI.GREEN}{len(case_base)}{UI.RESET}")

    UI.pause()

def view_all_cases() -> None:
    UI.header()
    UI.section("Archived Case Base Repository", "CASE VIEWER")
    for case in case_base:
        print(f"    {UI.MAGENTA}< {case['id']} >{UI.RESET}")
        print(f"        Features : {case['problem']}")
        print(f"        Solution : {UI.GREEN}{case['solution']}{UI.RESET}\n")
    UI.pause()

# MAIN MENU LOOP
def main_menu() -> None:
    while True:
        UI.header()
        print("\n  Select a module to execute:\n")
        print(UI.button("1", "View Knowledge Base (Part 1 - KR)"))
        print(UI.button("2", "Run Rule-Based Reasoning Engine (Part 2 -
RBR)"))
        print(UI.button("3", "Execute Case-Based Reasoning Pipeline (Part
3 - CBR 4R)"))
        print(UI.button("4", "View All Retained Cases in Memory"))
        print(UI.button("5", "Exit System"))
        print("\n    " + "-" * 50)

        choice = input(f"    {UI.CYAN}< INPUT >{UI.RESET} Enter Choice
[1-5]: ").strip()

        if choice == "1":
            display_knowledge_base()
        elif choice == "2":
            run_rule_based_reasoning()

```

```

elif choice == "3":
    run_case_based_reasoning()
elif choice == "4":
    view_all_cases()
elif choice == "5":
    UI.header()
    print(f"\n    {UI.GREEN}< OK > System session closed. Thank
you!{UI.RESET}\n")
    sys.exit(0)
else:
    print(f"\n    {UI.RED}< ERROR > Invalid choice! Please enter a
number from 1 to 5.{UI.RESET}")
    time.sleep(1)

if __name__ == "__main__":
    main_menu()

```

```

Welcome  TA4_Pardilla_Vergara.py x
TA4_Pardilla_Vergara.py > UI > header
1  import os
2  import sys
3  import time
4  from typing import Dict, List, Any, Tuple
5
6  # TERMINAL UI & AESTHETICS (BUTTON STYLING + ALIGNED BORDERS)
7  class UI:
8      CYAN = "\033[96m"
9      GREEN = "\033[92m"
10     YELLOW = "\033[93m"
11     MAGENTA = "\033[95m"
12     RED = "\033[91m"
13     BOLD = "\033[1m"
14     DIM = "\033[2m"
15     RESET = "\033[0m"
16
17     # Inner width between the vertical pipes: | ... 74 chars ... |
18     INNER_WIDTH = 74
19
20     @classmethod
21     def header(cls) -> None:
22         cls.clear()
23         w = cls.INNER_WIDTH
24
25         top_border = f"{cls.CYAN}+{'-' * w}+{cls.RESET}"
26         mid_border = f"{cls.CYAN}+{'-' * w}+{cls.RESET}"
27         bot_border = f"{cls.CYAN}+{'-' * w}+{cls.RESET}"
28
29         # Plain text strings for exact width calculation
30         t1_plain = "INTELLIGENT SYSTEMS: REASONING & INFERENCE ENGINE"
31         t2_plain = "CS0065 - Technical Assessment 4"
32         l3_plain = "  Programmed by : Dianna Francesca M. Pardilla & Justin D. Vergara"
33         l4_plain = "  Section      : CS0065 - AN41"
34
35         # Centered titles
36         t1_pad = (w - len(t1_plain)) // 2
37         t1_formatted = " " * t1_pad + f"{cls.BOLD}{t1_plain}{cls.RESET}" + " " * (w - len(t1_plain) - t1_pad)
38
39         t2_pad = (w - len(t2_plain)) // 2
40         t2_formatted = " " * t2_pad + f"{cls.DIM}{t2_plain}{cls.RESET}" + " " * (w - len(t2_plain) - t2_pad)
41
42         # Left-aligned info lines with exact trailing space padding
43         l3_formatted = f"  Programmed by : {cls.BOLD}Dianna Francesca M. Pardilla & Justin D. Vergara{cls.RESET}" + " " * (w - len(l3_plain))
44         l4_formatted = f"  Section      : {cls.YELLOW}CS0065 - AN41{cls.RESET}" + " " * (w - len(l4_plain))
45
46         # Render Header Box
47         print(top_border)

```

```
Welcome TA4_Pardilla_Vergara.py X
TA4_Pardilla_Vergara.py > UI > header
7 class UI:
21     def header(cls) -> None:
48         print(f"{cls.CYAN}|{cls.RESET}{t1_formatted}{cls.CYAN}|{cls.RESET}")
49         print(f"{cls.CYAN}|{cls.RESET}{t2_formatted}{cls.CYAN}|{cls.RESET}")
50         print(mid_border)
51         print(f"{cls.CYAN}|{cls.RESET}{l3_formatted}{cls.CYAN}|{cls.RESET}")
52         print(f"{cls.CYAN}|{cls.RESET}{l4_formatted}{cls.CYAN}|{cls.RESET}")
53         print(bot_border)
54
55     @classmethod
56     def section(cls, title: str, part_num: str) -> None:
57         """Displays section headers with a stylized < BUTTON > banner."""
58         tag_btn = f"{cls.MAGENTA}< {cls.BOLD}{part_num}{cls.RESET}{cls.MAGENTA} >{cls.RESET}"
59         title_btn = f"{cls.CYAN}< {cls.BOLD}{title.upper()}{cls.RESET}{cls.CYAN} >{cls.RESET}"
60         print(f"\n{tag_btn} — {title_btn}\n")
61
62     @classmethod
63     def button(cls, key: str, label: str) -> str:
64         """Returns a formatted button string for menu choices."""
65         badge = f"{cls.CYAN}< {cls.BOLD}{key}{cls.RESET}{cls.CYAN} >{cls.RESET}"
66         return f" {badge} {label}"
67
68     @classmethod
69     def pause(cls) -> None:
70         print(f"\n{cls.DIM}Press Enter to return to main menu...{cls.RESET}", end="")
71         input()
72
73     @classmethod
74     def clear(cls) -> None:
75         os.system("cls" if os.name == "nt" else "clear")
76
77 # PART 1: KNOWLEDGE REPRESENTATION (KR) SETUP
78 knowledge_base: Dict[str, Any] = {
79     # Category 1: Thermals & Environment
80     "core_temperature_celsius": 86,
81     "ambient_humidity_percent": 74,
82
83     # Category 2: Resource Utilization & Power
84     "cpu_load_percent": 96,
85     "cooling_pump_active": False,
86     "battery_charge_percent": 15,
87
88     # Category 3: System Health & Hardware Flags
89     "disk_io_errors": True,
90     "network_latency_ms": 350
91 }
92
93
```

```

141     deductions = []
142     for i, r in enumerate(self.rules, start=1):
143         result = r(facts)
144         if result:
145             deductions.append((f"RBR-{i:02d}", result))
146     return deductions
147
148
149 def run_rule_based_reasoning() -> None:
150     UI.header()
151     UI.section("Rule-Based Reasoning (Inference Cycle)", "PART 2")
152
153     engine = InferenceEngine([
154         rule_thermal_critical,
155         rule_cooling_failure,
156         rule_battery_preservation,
157         rule_disk_anomaly
158     ])
159
160     print(f" {UI.DIM}[*] Evaluating working memory across defined production rules...{UI.RESET}")
161     time.sleep(0.3)
162     actions = engine.run(knowledge_base)
163
164     print(f"\n {UI.BOLD}Inferred Prescriptive Actions ({len(actions)} matched):{UI.RESET}\n")
165     for rule_id, act in actions:
166         btn_tag = f" {UI.YELLOW}< {rule_id} >{UI.RESET}"
167         print(f" {btn_tag} {act}")
168
169     UI.pause()
170
171 # PART 3: CASE-BASED REASONING (CBR) - 4R CYCLE
172 case_base: List[Dict[str, Any]] = [
173     {
174         "id": "CASE-101",
175         "problem": {"overheating": True, "fan_spinning": False, "loud_buzzing": False, "blue_screen": False},
176         "solution": "Clean heatsink fins and replace blocked auxiliary fan motor."
177     },
178     {
179         "id": "CASE-102",
180         "problem": {"overheating": True, "fan_spinning": True, "loud_buzzing": True, "blue_screen": False},
181         "solution": "Inspect fan blades for physical friction; lubricate bearing housing."
182     },
183     {
184         "id": "CASE-103",
185         "problem": {"overheating": False, "fan_spinning": True, "loud_buzzing": False, "blue_screen": True},

```

```

186         "solution": "Run memory diagnostic tool (memtest86) to detect degraded RAM modules."
187     },
188     {
189         "id": "CASE-104",
190         "problem": {"overheating": True, "fan_spinning": False, "loud_buzzing": True, "blue_screen": True},
191         "solution": "Power supply unit (PSU) rail short-circuit; perform full board inspection."
192     }
193 ]
194
195
196 def compute_similarity(stored_problem: Dict[str, bool], incoming_problem: Dict[str, bool]) -> float:
197     keys = set(stored_problem.keys()) & set(incoming_problem.keys())
198     if not keys:
199         return 0.0
200     matches = sum(1 for k in keys if stored_problem[k] == incoming_problem[k])
201     return (matches / len(keys)) * 100.0
202
203
204 def retrieve(incoming: Dict[str, bool]) -> Tuple[Dict[str, Any], float]:
205     best_match = None
206     highest_score = -1.0
207     for case in case_base:
208         score = compute_similarity(case["problem"], incoming)
209         if score > highest_score:
210             highest_score = score
211             best_match = case
212     return best_match, highest_score
213
214
215 def run_case_based_reasoning() -> None:
216     UI.header()
217     UI.section("Case-Based Reasoning (4R Cycle)", "PART 3")
218
219     incoming_case = {
220         "overheating": True,
221         "fan_spinning": False,
222         "loud_buzzing": False,
223         "blue_screen": True
224     }
225
226     print(f" {UI.BOLD}Incoming Problem Vector:{UI.RESET}")
227     for k, v in incoming_case.items():
228         print(f"     • {k:<15}: {v}")
229
230     # Step 1: RETRIEVE
231     print(f"\n {UI.CYAN}< 1. RETRIEVE >{UI.RESET} Searching case library for closest pattern...")
232     time.sleep(0.3)

```

```

233 matched_case, score = retrieve(incoming_case)
234 print(f" -> Best Match : {UI.BOLD}< {matched_case['id']} >{UI.RESET} (Similarity: {UI.GREEN}{score:.1f}%{UI.RESET})")
235
236 # Step 2: REUSE
237 print(f"\n {UI.CYAN}< 2. REUSE >{UI.RESET} Retrieved Solution:")
238 reused_solution = matched_case["solution"]
239 print(f" -> \"{reused_solution}\"")
240
241 # Step 3: REVISE
242 print(f"\n {UI.CYAN}< 3. REVISE >{UI.RESET} Adjusting solution for unique symptom traits:")
243 print(" -> Enter adjusted solution (press Enter to keep default): ")
244 user_adj = input("      > ").strip()
245 revised_solution = user_adj if user_adj else f"{reused_solution} + [Addendum: Verify PSU voltage output]."
246 print(f" -> Final Solution: {UI.YELLOW}{revised_solution}{UI.RESET}")
247
248 # Step 4: RETAIN
249 print(f"\n {UI.CYAN}< 4. RETAIN >{UI.RESET} Archiving resolved case into memory base...")
250 new_id = f"CASE-{len(case_base) + 101}"
251 learned_case = {
252     "id": new_id,
253     "problem": incoming_case,
254     "solution": revised_solution
255 }
256 case_base.append(learned_case)
257 print(f" -> Case registered as {UI.BOLD}< {new_id} >{UI.RESET}.")
258 print(f" -> Total stored cases: {UI.GREEN}{len(case_base)}{UI.RESET}")
259
260 UI.pause()
261
262
263 def view_all_cases() -> None:
264     UI.header()
265     UI.section("Archived Case Base Repository", "CASE VIEWER")
266     for case in case_base:
267         print(f" {UI.MAGENTA}< {case['id']} >{UI.RESET}")
268         print(f"     Features : {case['problem']}")
269         print(f"     Solution : {UI.GREEN}{case['solution']}{UI.RESET}\n")
270     UI.pause()
271
272 # MAIN MENU LOOP
273 def main_menu() -> None:
274     while True:
275         UI.header()
276         print("\n Select a module to execute:\n")
277         print(UI.button("1", "View Knowledge Base (Part 1 - KR)"))

```

```

278     print(UI.button("2", "Run Rule-Based Reasoning Engine (Part 2 - RBR)"))
279     print(UI.button("3", "Execute Case-Based Reasoning Pipeline (Part 3 - CBR 4R)"))
280     print(UI.button("4", "View All Retained Cases in Memory"))
281     print(UI.button("5", "Exit System"))
282     print("\n " + "-" * 50)
283
284     choice = input(f" {UI.CYAN}< INPUT >{UI.RESET} Enter Choice [1-5]: ").strip()
285
286     if choice == "1":
287         display_knowledge_base()
288     elif choice == "2":
289         run_rule_based_reasoning()
290     elif choice == "3":
291         run_case_based_reasoning()
292     elif choice == "4":
293         view_all_cases()
294     elif choice == "5":
295         UI.header()
296         print(f"\n {UI.GREEN}< OK > System session closed. Thank you!{UI.RESET}\n")
297         sys.exit(0)
298     else:
299         print(f"\n {UI.RED}< ERROR > Invalid choice! Please enter a number from 1 to 5.{UI.RESET}")
300         time.sleep(1)
301
302
303 if __name__ == "__main__":
304     main_menu()

```

TECHNICAL ASSESSMENT 4 DOCUMENTATION

I. KNOWLEDGE BASE AND USE CASE DESCRIPTION

The automated Workstation and Server Diagnostic System, which is the main focus of our project, is intended to keep an eye on machine health and troubleshoot technical issues. The Knowledge Base serves as the system's real-time working memory.

- **Thermals & Environment:** Core temperature and ambient humidity are the three separate operational areas into which it stores seven facts.
- CPU load, cooling pump condition, and reserve battery levels are examples of resource utilization and power. Network latency and disk I/O errors are examples of system health and hardware flags.

Use Case & Reasoning Process:

- **Rule-Based Reasoning (RBR)** serves as an instantaneous emergency response system. Hardcoded rules immediately initiate automated countermeasures, such as CPU thermal throttling, fan overrides, and file system locking, to prevent irreversible hardware damage when live sensor values exceed safety thresholds.
- For complicated, multi-symptom failures, **Case-Based Reasoning (CBR)** serves as an intelligent tech support specialist. The system retrieves, adjusts, and stores new repair solutions when anomalous behavior (like blue screens or strange buzzing) occurs by comparing the symptom pattern with previous archived repairs.

II. SCREENSHOTS OF PROJECT EXECUTION

```
+-----+
|               INTELLIGENT SYSTEMS: REASONING & INFERENCE ENGINE               |
|               CS0065 - Technical Assessment 4                                |
+-----+
|  Programmed by : Dianna Francesca M. Pardilla & Justin D. Vergara            |
|  Section       : CS0065 - AN41                                              |
+-----+

Select a module to execute:

< 1 > View Knowledge Base (Part 1 - KR)
< 2 > Run Rule-Based Reasoning Engine (Part 2 - RBR)
< 3 > Execute Case-Based Reasoning Pipeline (Part 3 - CBR 4R)
< 4 > View All Retained Cases in Memory
< 5 > Exit System

_____
< INPUT > Enter Choice [1-5]: █
```

```
+-----+
|               INTELLIGENT SYSTEMS: REASONING & INFERENCE ENGINE               |
|               CS0065 - Technical Assessment 4                                |
+-----+
|  Programmed by : Dianna Francesca M. Pardilla & Justin D. Vergara            |
|  Section       : CS0065 - AN41                                              |
+-----+

< PART 1 > — < KNOWLEDGE REPRESENTATION (WORKING MEMORY) >

Telemetry Parameter | State / Value
+-----+
• Core Temperature Celsius | < 86 >
• Ambient Humidity Percent | < 74 >
• Cpu Load Percent         | < 96 >
• Cooling Pump Active      | < NORMAL: False >
• Battery Charge Percent   | < 15 >
• Disk Io Errors           | < CRITICAL: True >
• Network Latency Ms       | < 350 >

Press Enter to return to main menu... █
```

```
+-----+
|               INTELLIGENT SYSTEMS: REASONING & INFERENCE ENGINE               |
|               CS0065 - Technical Assessment 4                                |
+-----+
| Programmed by : Dianna Francesca M. Pardilla & Justin D. Vergara             |
| Section      : CS0065 - AN41                                                  |
+-----+

< PART 2 > — < RULE-BASED REASONING (INFERENCE CYCLE) >

[*] Evaluating working memory across defined production rules...

Inferred Prescriptive Actions (4 matched):

< RBR-01 > Thermal overload (>80°C) -> Trigger immediate CPU clock down-throttling.
< RBR-02 > Cooling pump offline under high load -> Force auxiliary high-RPM fan bypass.
< RBR-03 > Reserve power depleted (<20%) -> Terminate secondary GPU threads and background tasks.
< RBR-04 > I/O integrity flags asserted -> Mount main file-system partition as read-only.

Press Enter to return to main menu...|
```

```
+-----+
|               INTELLIGENT SYSTEMS: REASONING & INFERENCE ENGINE               |
|               CS0065 - Technical Assessment 4                                |
+-----+
| Programmed by : Dianna Francesca M. Pardilla & Justin D. Vergara             |
| Section      : CS0065 - AN41                                                  |
+-----+

< PART 3 > — < CASE-BASED REASONING (4R CYCLE) >

Incoming Problem Vector:
• overheating      : True
• fan_spinning     : False
• loud_buzzing     : False
• blue_screen      : True

< 1. RETRIEVE > Searching case library for closest pattern...
-> Best Match : < CASE-105 > (Similarity: 100.0%)

< 2. REUSE > Retrieved Solution:
-> "Clean heatsink fins and replace blocked auxiliary fan motor. + [Addendum: Verify PSU voltage output]."
```

```
< 3. REVISE > Adjusting solution for unique symptom traits:
-> Enter adjusted solution (press Enter to keep default):
>
-> Final Solution: Clean heatsink fins and replace blocked auxiliary fan motor. + [Addendum: Verify PSU voltage output]. + [Addendum: Verify PSU voltage output].

< 4. RETAIN > Archiving resolved case into memory base...
-> Case registered as < CASE-106 >.
-> Total stored cases: 6

Press Enter to return to main menu...|
```

```
+-----+
|               INTELLIGENT SYSTEMS: REASONING & INFERENCE ENGINE               |
|               CS0065 - Technical Assessment 4                                |
+-----+
|  Programmed by : Dianna Francesca M. Pardilla & Justin D. Vergara             |
|  Section       : CS0065 - AN41                                                 |
+-----+

< CASE VIEWER > — < ARCHIVED CASE BASE REPOSITORY >

< CASE-101 >
Features : {'overheating': True, 'fan_spinning': False, 'loud_buzzing': False, 'blue_screen': False}
Solution : Clean heatsink fins and replace blocked auxiliary fan motor.

< CASE-102 >
Features : {'overheating': True, 'fan_spinning': True, 'loud_buzzing': True, 'blue_screen': False}
Solution : Inspect fan blades for physical friction; lubricate bearing housing.

< CASE-103 >
Features : {'overheating': False, 'fan_spinning': True, 'loud_buzzing': False, 'blue_screen': True}
Solution : Run memory diagnostic tool (memtest86) to detect degraded RAM modules.

< CASE-104 >
Features : {'overheating': True, 'fan_spinning': False, 'loud_buzzing': True, 'blue_screen': True}
Solution : Power supply unit (PSU) rail short-circuit; perform full board inspection.

< CASE-105 >
Features : {'overheating': True, 'fan_spinning': False, 'loud_buzzing': False, 'blue_screen': True}
Solution : Clean heatsink fins and replace blocked auxiliary fan motor. + [Addendum: Verify PSU voltage output].

< CASE-106 >
Features : {'overheating': True, 'fan_spinning': False, 'loud_buzzing': False, 'blue_screen': True}
Solution : Clean heatsink fins and replace blocked auxiliary fan motor. + [Addendum: Verify PSU voltage output].
+ [Addendum: Verify PSU voltage output].

Press Enter to return to main menu...|
```

```
+-----+
|               INTELLIGENT SYSTEMS: REASONING & INFERENCE ENGINE               |
|               CS0065 - Technical Assessment 4                                |
+-----+
|  Programmed by : Dianna Francesca M. Pardilla & Justin D. Vergara             |
|  Section       : CS0065 - AN41                                                 |
+-----+

< OK > System session closed. Thank you!
```

III. COMPARATIVE REFLECTION

For me and Justin, building the RBR system was definitely the smoother road because it relies on simple "if this happens, do that" conditions that make sense right away. You just lay down the ground rules, run the checks against your facts, and it instantly triggers the answer. With CBR, things got tricky fast because we had to figure out how to compare traits, rank similarity scores, and guide the system through the entire 4R flow of recalling an old case, tweaking the fix, and keeping it saved in our case bank. A classic real world use for RBR is an automated ATM or banking app checking your balance before letting you withdraw money since it is strictly by the book with zero guesswork.

Where CBR really shines is when problems get chaotic or unpredictable. Real life issues rarely fit neatly into textbook rules, and while RBR simply fails if an exact condition is missing, CBR is smart enough to find the nearest past scenario that looks somewhat close and improvise a fix. That is also why it handles ever changing data so well because it grows on its own simply by banking each new case it encounters, without needing us to rewrite new rules every single time. Think of how hospital triage desks or tech repair shops work, where doctors and technicians constantly look back at previous patients or glitchy gadgets they handled in the past to quickly crack a tricky new problem today.